

AWS VPC Security Groups

Muhammad Ammar Rana

github.com/xsuwittch

1. Objective

Theoretical aspects of security groups are one area of the topic. Practical learning on actually seeing a REJECT in the logs sink the moment you break a rule is another area of the topic. The lab discussed in this context is a live demonstration of the fundamental concepts SG reference, SG behavior, stateful, and direction but reveals these concepts using real traffic and CloudWatch logs. In this report, all access decisions are corroborated with VPC Flow Logs; nothing is taken for granted.

2. Architecture

Three EC2 instances across a public and private subnet. Named after Game of Thrones characters because the access relationships between them map almost perfectly to the show.

Instance	Subnet	Role	Public IP
jammie	Public	Bastion single entry point into the setup	Yes
cersei	Public	Public instance, only reachable through jammie	Yes
tyrion	Private	No public IP, only reachable through jammie	No

2.1 Access Matrix

What the security group rules should enforce, before any traffic was tested:

Source	Destination	Result
Laptop → jammie		ALLOW
jammie → cersei		ALLOW
jammie → tyrion		ALLOW
cersei → tyrion		BLOCK
Laptop → cersei		BLOCK
Laptop → tyrion		BLOCK

3. Security Group Configuration

3.1 Why SG-to-SG Instead of IP Rules

IP addresses are fragile as trust anchors. Auto scaled instances get new IPs. Deployments change addresses. Using SG IDs as sources means AWS checks whether the connecting instance has the right SG attached not where its IP came from. Any instance wearing the right badge gets through, automatically, without touching any rule.

The screenshot displays the AWS Management Console interface for Security Groups. The top section, titled 'Security Groups (1/5)', shows a list of security groups. The second group, 'sg-07bc69639c7fd90aa' named 'Jammie, a good brother', is selected. Below this, the 'Inbound rules (1)' section shows a single rule with the source set to 'sg-06bed4cab675cb1ec' (the 'Jammie' group), allowing SSH access on port 22.

Name	Security group ID	Security group name	VPC ID	Description	Owner
-	sg-0be2cc4b6d95b622a	launch-wizard-2	vpc-0dbfff125a0957b0e	launch-wizard-2 created 2026-06-20T1...	660852786770
☒	sg-07bc69639c7fd90aa	Jammie, a good brother	vpc-0dbfff125a0957b0e	Cersei may not SSH tyrion	660852786770
-	sg-00db35de85243880d	default	vpc-0c7ce6cd4e112da88	default VPC security group	660852786770
-	sg-094cff157f517a603	default	vpc-0dbfff125a0957b0e	default VPC security group	660852786770
-	sg-0fac54210c19a1b41	launch-wizard-1	vpc-0dbfff125a0957b0e	launch-wizard-1 created 2026-06-20T1...	660852786770

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
-	sg-06bed4cab675cb1ec	-	SSH	TCP	22	sg-0be2cc4b6d95b622...	Jammie Can SSH both ...

Figure 1 jammie's SG ID set as the trusted source on cersei and tyrion. Note the description: "Cersei may not SSH tyrion."

4. Test Results

Each scenario from the access matrix was tested and the result verified in CloudWatch Flow Logs.

4.1 Direct SSH from Laptop to Cersei Blocked

Tried SSHing straight into cersei from the local machine, skipping the bastion. The SG has no rule for external IPs. Connection timed out before it reached the instance.

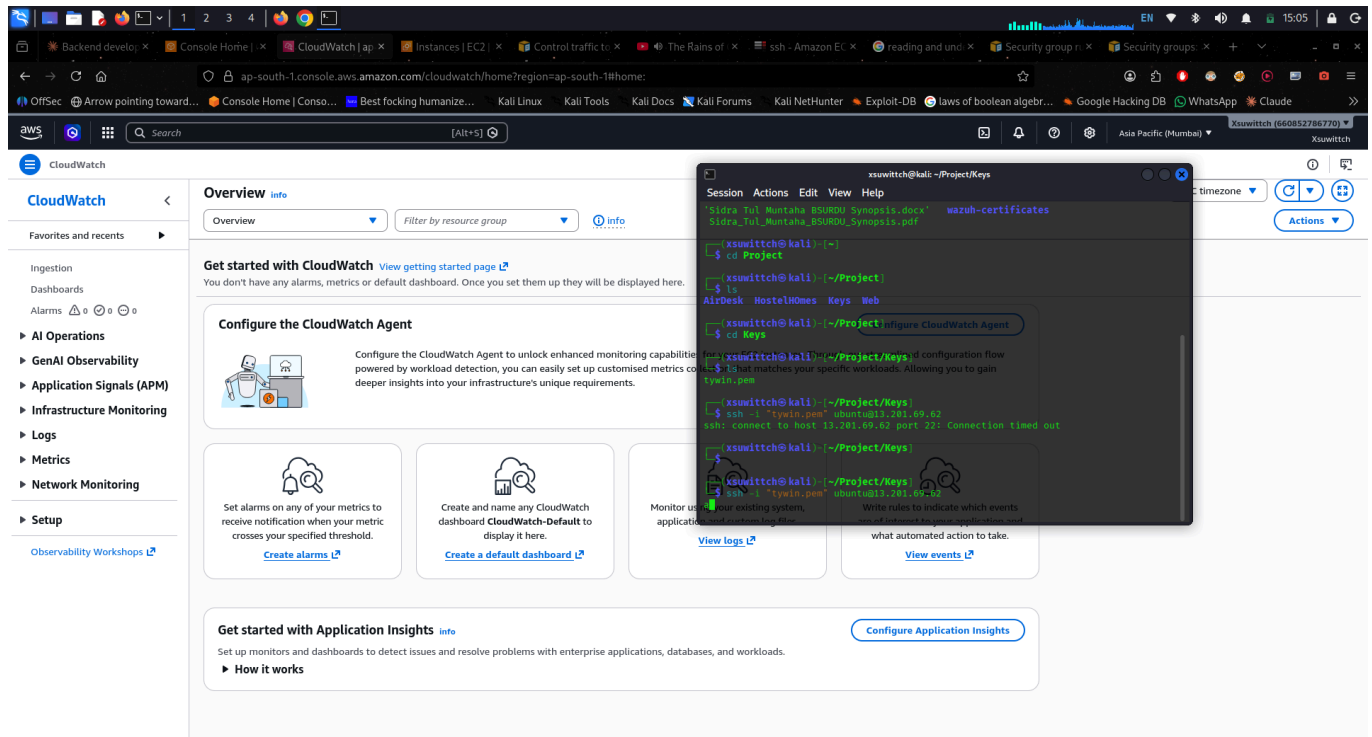


Figure 2 Connection timed out. The SG dropped the packet before any handshake could happen.

Timestamp	Message
2026-06-21T10:04:55.000Z	2 660852786770 eni-03b078b51838786f2 154.192.43.167 10.0.5.109 2941 22 6 8 480 1782036295 1782036323 REJECT OK
2026-06-21T10:04:55.000Z	2 660852786770 eni-03b078b51838786f2 154.192.43.167 10.0.5.109 2976 22 6 1 60 1782036295 1782036323 REJECT OK
2026-06-21T10:05:22.000Z	2 660852786770 eni-03b078b51838786f2 154.192.43.167 10.0.5.109 2992 22 6 1 60 1782036322 1782036350 REJECT OK
2026-06-21T10:05:59.000Z	2 660852786770 eni-03b078b51838786f2 154.192.43.167 10.0.5.109 1051 22 6 1 60 1782036359 1782036380 REJECT OK

Figure 3 Flow Logs confirming the REJECT. Source 154.192.43.167 (local machine) hitting cersei's private IP on port 22.

4.2 Jammie SSHing into Cersei Allowed

From inside jammie, SSH into cersei using its private IP. Jammie carries the trusted SG so cersei's rule lets it through.

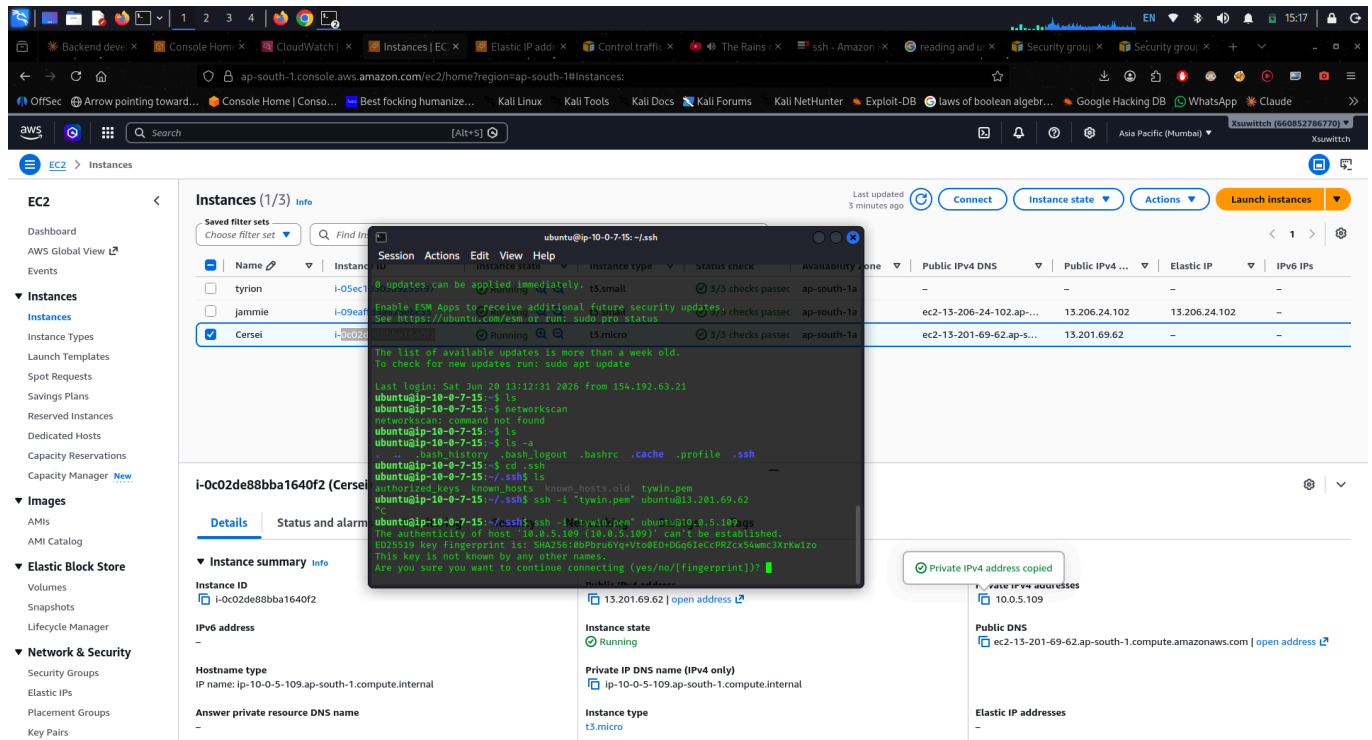


Figure 4 Successful SSH from jammie into cersei on private IP 10.0.5.109.

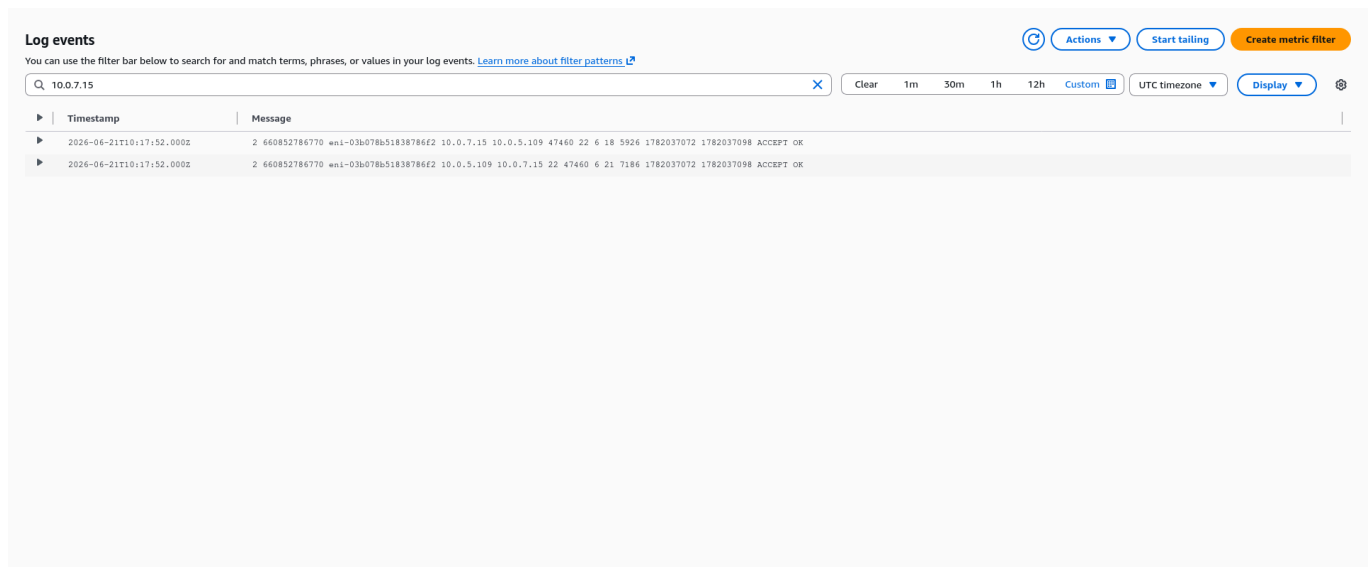


Figure 5 ACCEPT in both directions. Second line is return traffic auto-allowed stateful behavior proven in the logs.

4.3 Jammie SSHing into Tyrion Allowed

Tyrion is in a private subnet with no public IP. Two hops required: local machine to jammie, then jammie to tyrion via private IP.

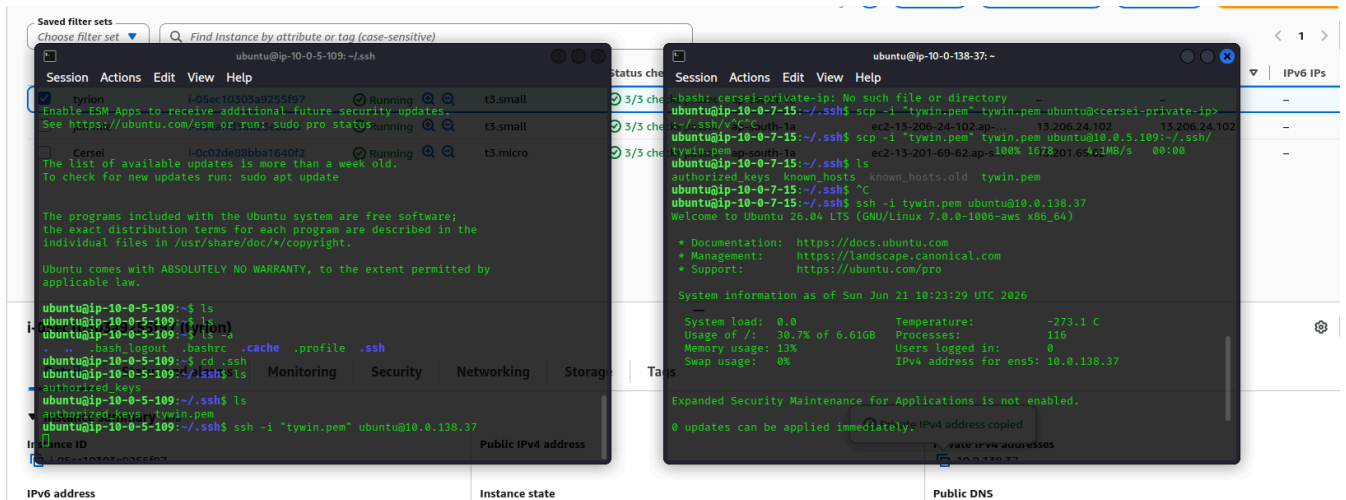


Figure 6 Both sessions open: cersei (left) and tyrion (right) accessible from jammie.

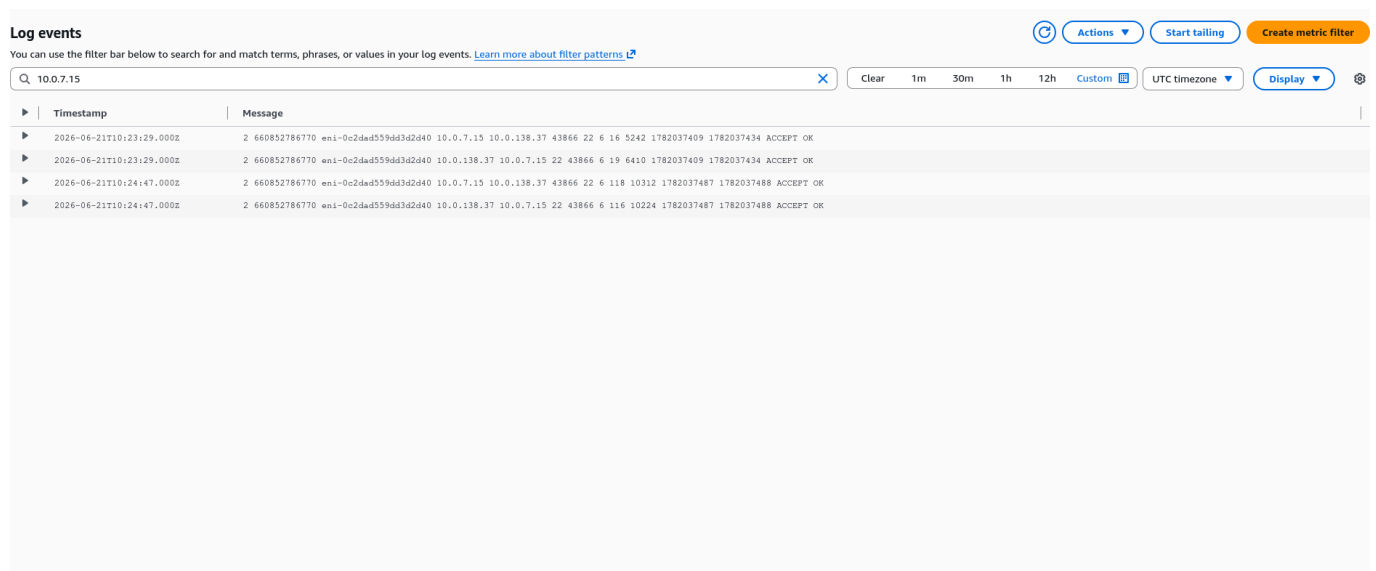


Figure 7 Tyrion's logs filtered on jammie's IP (10.0.7.15). ACCEPT on both request and response.

4.4 Cersei Attempting SSH into Tyrion Blocked

From inside cersei, tried to SSH into tyrion. Same VPC, adjacent subnets. Still blocked because cersei's SG is not in tyrion's trust list. Being in the same network does not grant access.

Timestamp	Message
2026-06-21T10:23:18.000Z	2 660852786770 eni-0c2dad559dd3d2d40 10.0.5.109 10.0.138.37 46736 22 6 7 420 1782037398 1782037407 REJECT OK
2026-06-21T10:23:29.000Z	2 660852786770 eni-0c2dad559dd3d2d40 10.0.5.109 10.0.138.37 46736 22 6 3 180 1782037409 1782037434 REJECT OK
2026-06-21T10:24:25.000Z	2 660852786770 eni-0c2dad559dd3d2d40 10.0.5.109 10.0.138.37 46736 22 6 1 60 1782037465 1782037466 REJECT OK

Figure 8 Tyrion's logs showing cersei's REJECT. Source 10.0.5.109 hitting port 22 blocked at the SG level.

5. Key Concepts

5.1 Default Deny

SGs block everything unless explicitly allowed. No rule means no access, full stop. The direct SSH attempt from the local machine to cersei timed out without even getting a TCP reset back the SG dropped it before the connection established.

5.2 SG-to-SG Referencing

Using a SG ID as a source is the right pattern for multi-tier AWS architecture. It is identity-based, not address-based. IPs change. SG IDs don't. New instance with the right SG attached? Gets access automatically.

5.3 Stateful Behavior

Once a connection is permitted in one direction, the return traffic gets through automatically. The logs show this clearly every accepted SSH session produced two ACCEPT entries, request and response. No separate outbound rule needed.

NACLs don't work this way. They are stateless and evaluate every packet in both directions independently. This distinction matters when debugging access issues.

5.4 Defence in Depth

Tyrion has two independent layers. A private subnet means there's no route to it from the internet. The SG adds a second check only jammie's SG gets through. Both have to fail simultaneously for an attacker to reach tyrion.

5.5 Bots Are Instant

Within minutes of cersei getting a public IP, automated scanners from Shodan, Censys, and various botnets were already probing random ports. All blocked by the SG. This is visible in the flow logs and is exactly why default deny exists and why port 22 should never be open to 0.0.0.0/0 on anything except a dedicated bastion.